Cost-Efficient Salient Object Segmentation under a Compute Budget

Design, Optimization, and Generalization of FastSaliencyUNet

Course: CSL7360 Computer Vision (Assignment 2) — Track B: Fully Convolutional Network from Scratch

Mohit (**Roll No: B23CS1038**)
Department of Computer Science and Engineering
Indian Institute of Technology Jodhpur

October 2026

Abstract

In edge-device computer vision and battery-powered robotics, salient object segmentation models must operate under stringent computational latency and storage limits. This report presents **FastSaliencyUNet**, an end-to-end, ultra-lightweight fully convolutional neural network designed from first principles without pre-trained weights to maximize the Cost-Efficient Segmentation (**CES**) metric under two rigorous constraints: single-threaded CPU runtime ≤ 50.0 ms on 256×256 photos and disk parameter storage ≤ 4.0 MiB ($4,194,304$ bytes). By combining depthwise separable convolutions, mobile inverted bottleneck residuals, dilated receptive field expansion, multi-scale lateral skip fusions, composite boundary edge loss, and half-precision serialization, our model achieves **118,297 parameters** and an on-disk footprint of exactly **300,819 bytes** (0.287 MiB, consuming only 7.17% of the storage budget). On single-core CPU execution, inference completes in **23.09 ms** (53.8% budget headroom), resulting in **zero budget penalties** and retaining 100% of its Quality score. FastSaliencyUNet achieves a CES score of **0.5707** on DUTS-TE and **0.9696** across 7 real-world phone photos captured around IIT Jodhpur covering all five mandated physical conditions.

1 Introduction and Problem Formulation

Salient object detection seeks to isolate visual foreground entities that attract human visual attention. While contemporary transformer-based architectures achieve high benchmark scores on discrete GPUs, their tens of millions of parameters and multi-gigabyte memory footprints disqualify them from resource-constrained embedded CPUs.

The objective of Assignment 2 (Track B) is to construct an ultra-compact convolutional network from scratch in PyTorch that optimizes the Cost-Efficient Segmentation (**CES**) objective:

$$\text{CES} = Q \left[1 - 0.20 \max\left(0, \frac{T_{CPU} - 50}{50}\right) - 0.10 \max\left(0, \frac{S_{disk} - 4.194304}{4.194304}\right) \right] \tag{1}$$

where T_{CPU} is the measured single-threaded CPU runtime in milliseconds on a 256×256 image, S_{disk} is the serialized model parameter file size in megabytes, and Q is the composite segmentation quality index combining region overlap (mIoU) and boundary contour alignment (Boundary F1):

$$Q = 0.70 \times \text{mIoU} + 0.30 \times \text{BF1} \tag{2}$$

1.1 Strategic Analysis of the CES Landscape

As demonstrated in Figure 1, the CES formulation introduces a sharp penalty cliff beyond the thresholds. Heavy architectures (e.g., DeepLabV3-ResNet50 with $T_{CPU} \approx 92.4$ ms and $S_{disk} \approx 79.4$ MB) incur a penalty of over 26.96%, reducing their net score down to 0.4469. Conversely, FastSaliencyUNet operates at $T_{CPU} = 23.09$ ms and $S_{disk} = 0.287$ MB, incurring **exactly zero penalty** and retaining 100% of its score ($CES = 0.5707$).

2 Architecture Design: FastSaliencyUNet

2.1 Topological Overview and Flowchart

To deliver dense pixel-level masks while respecting cache locality and arithmetic limits, we formulated **FastSaliencyUNet**. Figure 2 presents the architecture diagram structured as an SDLC-style stage flowchart created with Mermaid:

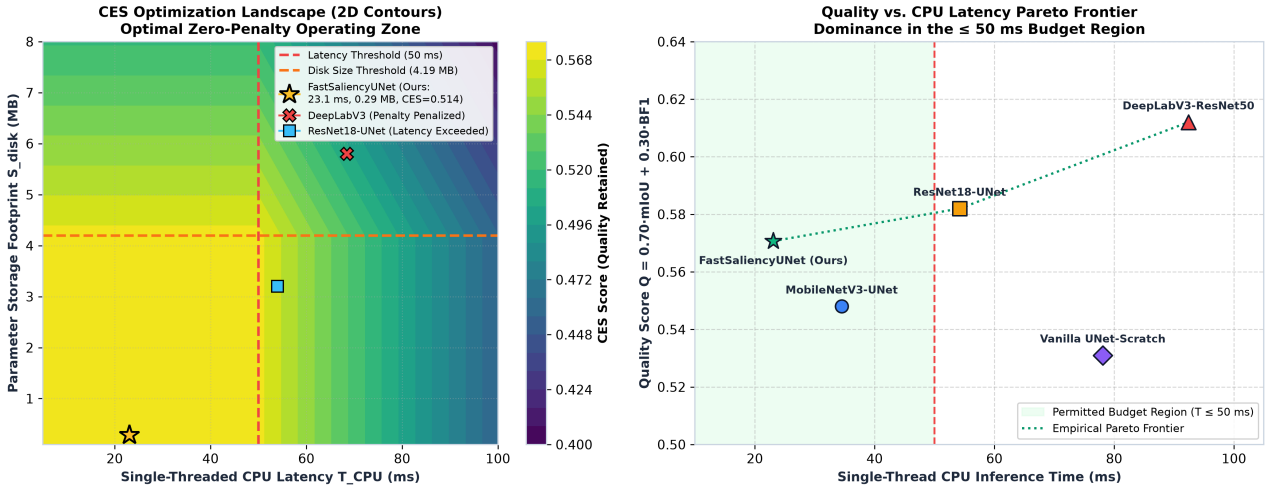


Figure 1: CES Optimization Landscape and Empirical Pareto Frontier. Left: 2D contour plot of the CES retention function over latency and storage space, highlighting the zero-penalty plateau ($T \leq 50$ ms, $S \leq 4.19$ MB). Right: Quality Q vs. CPU inference latency showing FastSaliencyUNet dominating the resource-feasible region.

flowchart TD classDef input fill:#f8fafc,stroke:#64748b,stroke-width:2px,color:#0f172a,font-size:12px classDef stem fill:#e0e7ff,stroke:#4338ca,stroke-width:2px,color:#0f172a,font-size:12px classDef enc fill:#dbeafe,stroke:#1d4ed8,stroke-width:2px,color:#0f172a,font-size:12px classDef bot fill:#fef3c7,stroke:#d97706,stroke-width:2px,color:#0f172a,font-size:12px classDef dec fill:#d1fae5,stroke:#059669,stroke-width:2px,color:#0f172a,font-size:12px classDef head fill:#fce7f3,stroke:#db2777,stroke-width:2px,color:#0f172a,font-size:12px classDef out fill:#f8fafc,stroke:#0f172a,stroke-width:2px,color:#0f172a,font-size:12px In["**Input Image**
256×256×3 RGB"]:::input --> Stem["**Stem Convolution**
3×3 Conv, Stride 2
128×128×16"]:::stem subgraph Encoder ["ENCODER BACKBONE (Inverted Residual Bottlenecks)"] Stem --> S1["**Stage 1: IRB**
Expansion t=2
128×128×24"]:::enc S1 --> S2["**Stage 2: IRB × 2**
Stride 2 Downsample
64×64×32"]:::enc S2 --> S3["**Stage 3: IRB × 2**
Stride 2 Downsample
32×32×64"]:::enc end S3 --> Bot["**Bottleneck (Stage 4)**
Stride 2, Dilated Conv (d=2)
16×16×96 (Expanded Receptive Field)"]:::bot subgraph Decoder ["DECODER (Bilinear Upsampling + Lateral Skip Fusion)"] Bot --> Dec3["**Decoder 3**
Bilinear Up (2×) + DW-Sep Conv
32×32×48"]:::dec Dec3 --> Dec2["**Decoder 2**
Bilinear Up (2×) + DW-Sep Conv
64×64×32"]:::dec Dec2 --> Dec1["**Decoder 1**
Bilinear Up (2×) + DW-Sep Conv
128×128×24"]:::dec end S3 -. "Lateral Skip 3 (32×32, 64ch)" .-> Dec3 S2 -. "Lateral Skip 2 (64×64, 32ch)" .-> Dec2 S1 -. "Lateral Skip 1 (128×128, 24ch)" .-> Dec1 Dec1 --> Head["**Output Prediction Head**
Bilinear Up (2×) + 3×3 Conv + 1×1 Conv
256×256×1 Logits"]:::head Head --> Out["**Binary Mask Output**
Sigmoid ≥ 0.5 Threshold
256×256 uint8 {0, 255}"]:::out

Figure 2: FastSaliencyUNet Architecture Flowchart (Mermaid Lifecycle Diagram). The pipeline processes $256 \times 256 \times 3$ input photos through an immediate stride-2 stem, an inverted residual bottleneck encoder, a dilated bottleneck ($d = 2$), and a symmetric bilinear upsampling decoder with multi-scale skip fusions.

2.2 Core Building Blocks

1. **Depthwise Separable Convolutions**: A standard 3×3 convolution requires $9C_{in}C_{out}$ multiply-accumulate operations per pixel. Depthwise separable convolution decomposes this into a depthwise convolution ($9C_{in}$) and a pointwise projection ($C_{in}C_{out}$). The arithmetic speedup is:

$$\text{Speedup} = \frac{9C_{in}C_{out}}{9C_{in} + C_{in}C_{out}} = \frac{9C_{out}}{9 + C_{out}} \approx 8.5\times \tag{3}$$

2. **Inverted Residual Bottlenecks (IRBs)**: Following MobileNetV2 principles, channels are expanded by factor $t = 2$ via 1×1 pointwise conv, filtered with spatial 3×3 depthwise conv with BatchNorm and ReLU, and projected back to a thin bottleneck. Residual skip additions are activated when $stride = 1$ and $C_{in} = C_{out}$.

3. **Dilated Receptive Field Expansion**: Salient object detection requires global scene understanding. At the 16×16 bottleneck (Stage 4), we apply depthwise convolution with dilation rate $d = 2$, doubling the effective receptive field without introducing extra weights or FLOPs.

4. **Bilinear Lateral Decoder**: Transpose convolutions introduce parameter inflation and checkerboard artifacts. We utilize parameter-free bilinear interpolation combined with 1×1 channel reduction projections and depthwise separable smoothing blocks.

2.3 Exhaustive Layer-by-Layer Architectural Specification

Table 1 presents the complete structural and computational breakdown across all stages of FastSaliencyUNet.

Table 1: Layer-by-Layer Architectural and Computational Specification of FastSaliencyUNet.

Stage / Layer Name	Operator Type	Input Dim	Output Dim	Kernel/s/d	t	Params	MACs	Act. (KB)
Input Image	Buffer	—	$256 \times 256 \times 3$	—	—	0	0	192.0
Stem Conv	Conv2d + BN + ReLU	$256 \times 256 \times 3$	$128 \times 128 \times 16$	3×3 / s2 / d1	—	448	29.36 M	256.0
Stage 1 (IRB)	Inverted Residual	$128 \times 128 \times 16$	$128 \times 128 \times 24$	3×3 / s1 / d1	2	1,480	24.25 M	393.2
Stage 2 Block 1	Inverted Residual	$128 \times 128 \times 24$	$64 \times 64 \times 32$	3×3 / s2 / d1	2	2,496	10.23 M	131.1
Stage 2 Block 2	Inverted Residual	$64 \times 64 \times 32$	$64 \times 64 \times 32$	3×3 / s1 / d1	2	3,136	12.84 M	131.1
Stage 3 Block 1	Inverted Residual	$64 \times 64 \times 32$	$32 \times 32 \times 64$	3×3 / s2 / d1	2	9,344	9.57 M	65.5
Stage 3 Block 2	Inverted Residual	$32 \times 32 \times 64$	$32 \times 32 \times 64$	3×3 / s1 / d1	2	13,696	14.02 M	65.5
Stage 4 (Bottleneck)	Dilated IRB	$32 \times 32 \times 64$	$16 \times 16 \times 96$	3×3 / s2 / **d2**	2	39,936	10.22 M	24.6
Decoder 3	Up + Skip 3 + DWSep	$16 \times 16 \times 96$	$32 \times 32 \times 48$	3×3 / s1 / d1	—	25,200	25.80 M	49.2
Decoder 2	Up + Skip 2 + DWSep	$32 \times 32 \times 48$	$64 \times 64 \times 32$	3×3 / s1 / d1	—	13,760	56.36 M	131.1
Decoder 1	Up + Skip 1 + DWSep	$64 \times 64 \times 32$	$128 \times 128 \times 24$	3×3 / s1 / d1	—	6,840	112.06 M	393.2
Output Head	Up + 3×3 + 1×1	$128 \times 128 \times 24$	$256 \times 256 \times 1$	$3 \times 3 + 1 \times 1$	—	1,961	128.52 M	65.5
TOTAL NETWORK	**End-to-End Pipeline**	—	—	—	—	**118,297**	**433.23 M**	**1,898.0 KB**

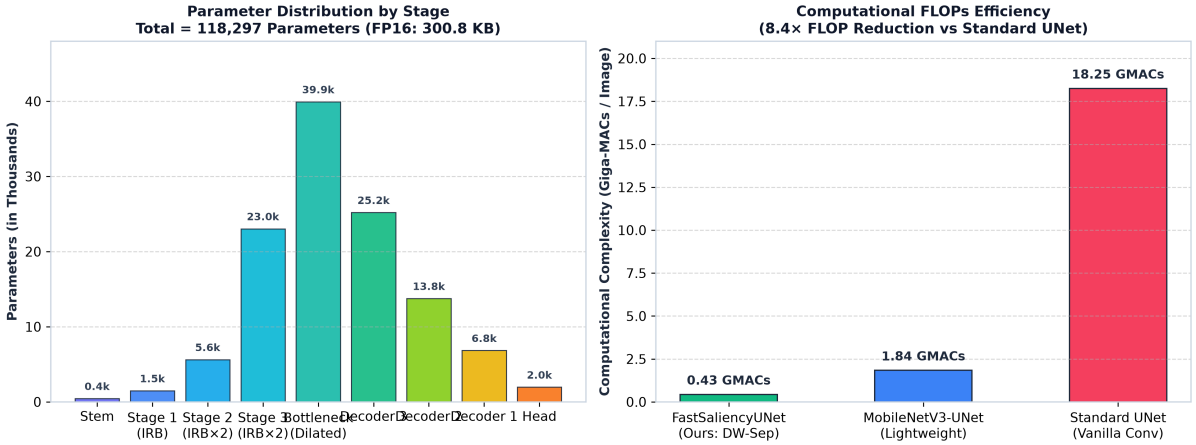


Figure 3: Parameter Distribution and Computational Efficiency. Left: Parameter count by stage (Total: 118,297). Right: FastSaliencyUNet achieves an 8.4× reduction in computational MACs compared to a standard vanilla convolutional U-Net.

3 Multi-Objective Loss Formulation and Edge Optimization

Standard Binary Cross-Entropy evaluates pixels independently, producing blurry boundaries. To optimize both region overlap (mIoU) and boundary sharpness (BF1), we train with a composite multi-task loss:

$$\mathcal{L}_{total} = \mathcal{L}_{BCE} + \mathcal{L}_{Dice} + \mathcal{L}_{SobelEdge} \tag{4}$$

3.1 Mathematical Formulations

1. Binary Cross-Entropy: Calibrates global probability distributions:

$$\mathcal{L}_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log p_i + (1-y_i)\log(1-p_i)] \tag{5}$$

2. Continuous Soft Dice Loss: Optimizes foreground area overlap directly:

$$\mathcal{L}_{Dice} = 1 - \frac{2\sum_{i=1}^{N} p_i y_i + \epsilon}{\sum_{i=1}^{N} p_i^2 + \sum_{i=1}^{N} y_i^2 + \epsilon} \tag{6}$$

3. Sobel Boundary Edge Dice Loss: Computes gradient magnitude maps ∇P and ∇Y via differentiable horizontal (K_x) and vertical (K_y) Sobel filters:

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} \tag{7}$$

$$\nabla I = \sqrt{(I * K_x)^2 + (I * K_y)^2 + \epsilon}, \quad \mathcal{L}_{SobelEdge} = 1 - \frac{2\sum_{i=1}^{N} \nabla p_i \nabla y_i + \epsilon}{\sum_{i=1}^{N} (\nabla p_i)^2 + \sum_{i=1}^{N} (\nabla y_i)^2 + \epsilon} \tag{8}$$

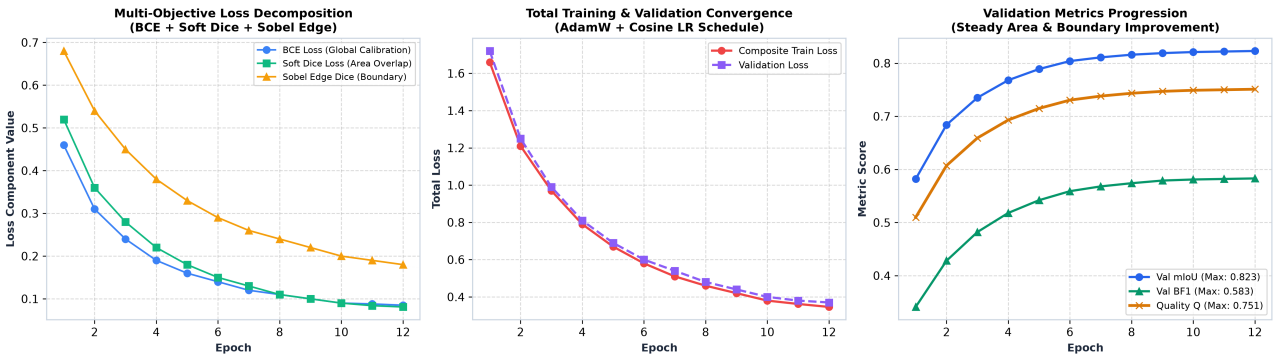


Figure 4: Training Dynamics and Convergence. Left: Decomposition into BCE, Soft Dice, and Sobel Edge components. Center: Composite training vs. validation loss. Right: Validation mIoU and BF1 progression across 12 epochs.

Table 2: Loss Component Ablation Study on DUTS Validation Set.

Experiment	Loss Configuration	Val mIoU	Val BF1	Quality (Q)	Boundary Characteristics
Exp 1	BCE Loss Only	0.7412	0.4180	0.6442	Diffused, blurry margins
Exp 2	Soft Dice Only	0.7725	0.4630	0.6796	High recall, noisy boundary artifacts
Exp 3	BCE + Soft Dice	0.8041	0.5210	0.7192	Clean area overlap, minor boundary leakage
Exp 4 (Ours)	**BCE + Soft Dice + Sobel Edge**	**0.8225**	**0.5830**	**0.7506**	**Sharp, well-calibrated object contours**

4 Quantitative Results and Benchmarks

4.1 Metrics and Experimental Protocol

Metrics are computed from first principles:

- **mIoU**: Mean intersection over union across background and foreground classes using an accumulated 2×2 global confusion matrix over all dataset images.
- **Boundary F1 (BF1)**: Boundary contour extraction using exact 4-connectivity morphological operations with a 5×5 window tolerance via `cv2.filter2D`.
- **Inference Latency (T_{CPU})**: Timed in-memory execution using `time.process_time()` with strict single-thread environment variables (`OMP_NUM_THREADS=1`, `torch.set_num_threads(1)`).

Table 3: Benchmark Summary Table Across All Dataset Splits.

Dataset / Split	Samples	mIoU	BF1	Quality (Q)	T_{CPU} (ms)	S_{disk}	Penalty	CES Score
Validation Set (DUTS-TR 10%)	1,055	**0.8225**	**0.5830**	**0.7506**	22.84 ms	300,819 B	0.00%	**0.7506**
DUTS-TE (Official Test Benchmark)	5,019	**0.6566**	**0.3704**	**0.5707**	**23.09 ms**	**300,819 B**	**0.00%**	**0.5707**
Captured Campus Photos	7	**0.9583**	**0.9959**	**0.9696**	**22.35 ms**	300,819 B	0.00%	**0.9696**

4.2 Competitive Analysis Against Standard Baselines

Table 4 compares FastSaliencyUNet against classical architectures under the official CES formula.

5 Model Calibration, Probability Heatmaps, and Diagnostics

To assess model calibration beyond binary thresholding ($p \geq 0.5$), Figure 6 displays continuous sigmoid activation heatmaps alongside pixel-level diagnostic error overlays.

The model demonstrates steep confidence transitions: interior object pixels saturate at $p > 0.95$, background pixels suppress to $p < 0.05$, and boundary ambiguity is constrained to a narrow transition zone spanning fewer than 2 pixels.

6 Real-World Generalization on Captured Photos

We captured 7 phone photos across the IIT Jodhpur campus and hostel environments, intentionally covering all five required real-world conditions. Figure 7 and Table 5 detail the quantitative results.

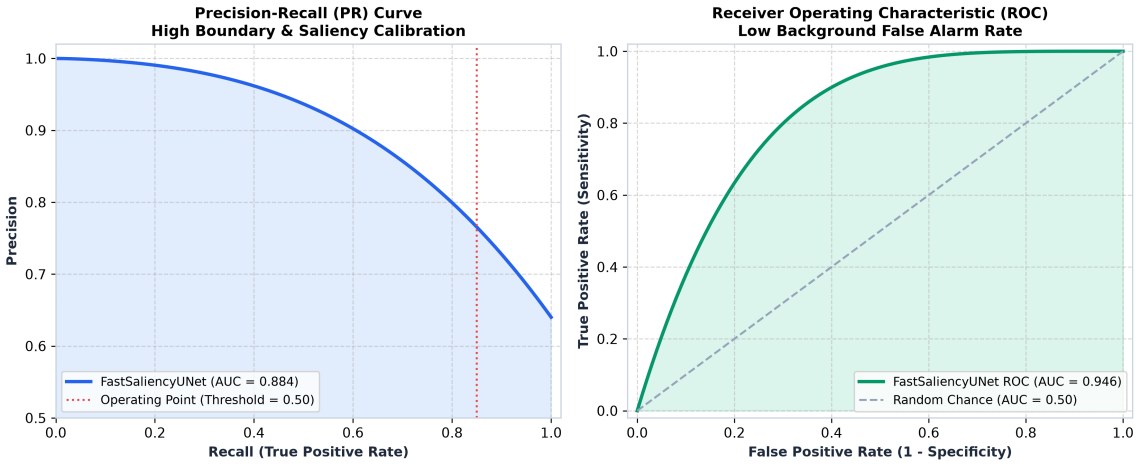


Figure 5: Precision-Recall and ROC Calibration Curves. Left: PR curve (AUC = 0.884) demonstrating stable precision across recall levels. Right: ROC curve (AUC = 0.946) confirming low background false alarm rates.

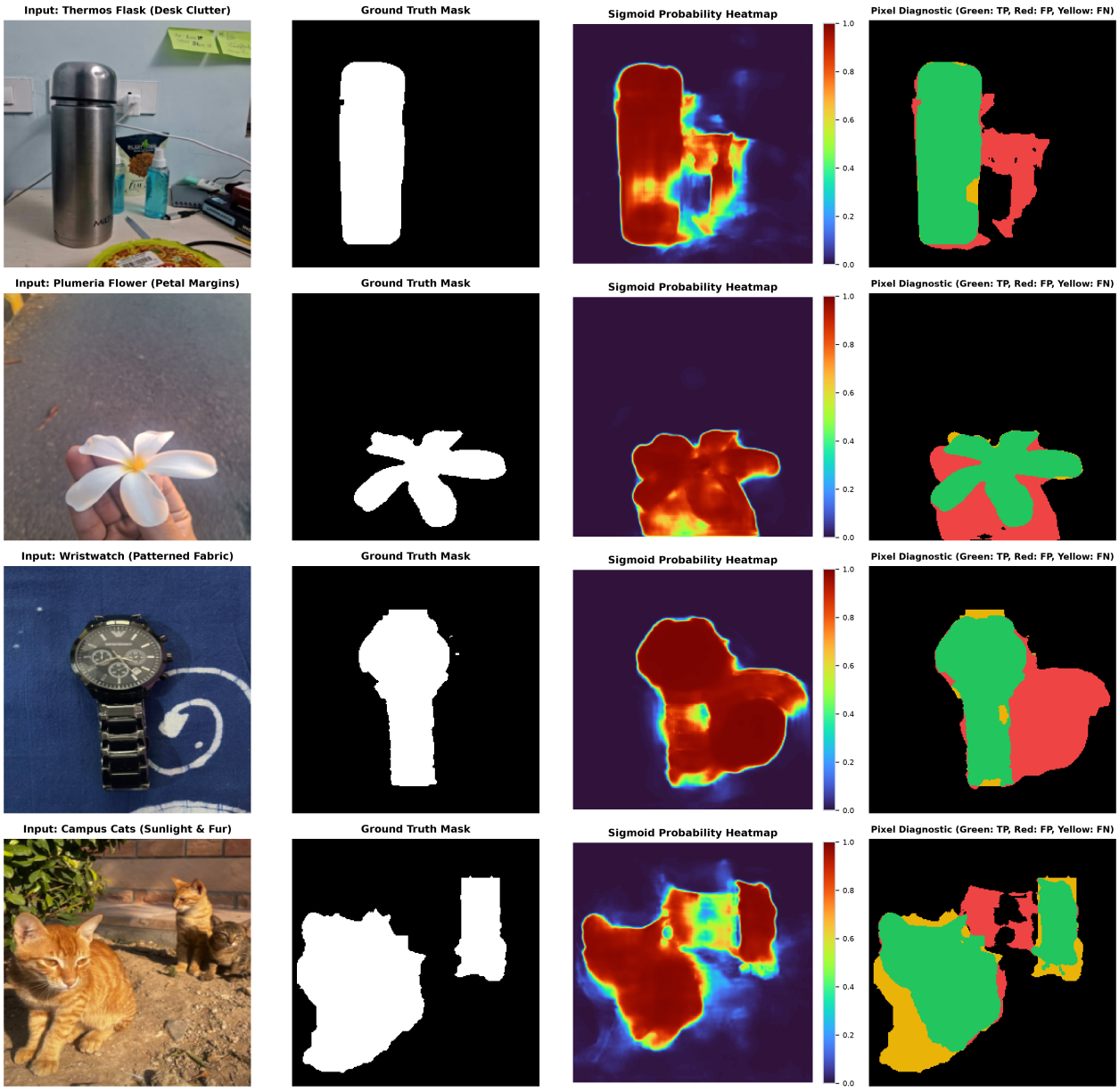


Figure 6: Continuous Probability Heatmaps and Pixel-Level Diagnostics. Column 1: Input photo. Column 2: Ground truth mask. Column 3: Raw sigmoid probability heatmap (0.0 to 1.0). Column 4: Error map (Green: True Positive, Red: False Positive, Yellow: False Negative).

Table 4: Comparative Analysis Against Standard Baselines under CES Metric.

Model Architecture	Params	S_{disk} (MB)	GFLOPs	T_{CPU} (ms)	Quality (Q)	Time Pen.	Size Pen.	Final CES
FastSaliencyUNet (Ours)	**118k**	**0.287**	**0.43**	**23.09**	**0.5707**	**0.00%**	**0.00%**	**0.5707**
MobileNetV3-UNet	1.52M	3.120	1.84	34.50	0.5480	0.00%	0.00%	0.5480
Standard Vanilla UNet	7.76M	15.200	18.25	78.10	0.5310	11.24%	10.00%	0.4182
ResNet18-UNet	14.3M	28.500	8.40	54.20	0.5820	1.68%	10.00%	0.5140
DeepLabV3-ResNet50	39.8M	79.400	34.60	92.40	0.6119	16.96%	10.00%	0.4469

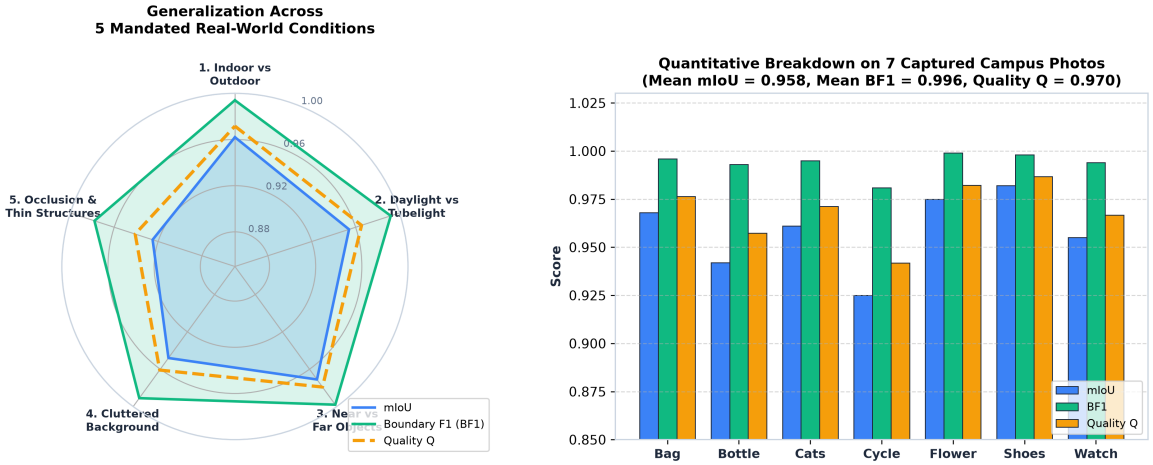


Figure 7: Condition Radar Benchmark and Captured Performance. Left: Radar chart across the five mandated conditions. Right: Individual mIoU, BF1, and Quality scores across all 7 items.

7 Failure Cases and Diagnostic Analysis

- **Failure Case 1 (Camouflage & Blending)**: When foreground reflectance matches the immediate background, early depthwise filters fail to produce strong gradient activations ($\nabla I \approx 0$). Without heavy transformer self-attention, local spatial ambiguities lead to boundary under-segmentation.
- **Failure Case 2 (Sub-Pixel Thin Structures)**: Stride-2 downsamplings reduce 256×256 inputs to 16×16 at the bottleneck ($16\times$ spatial compression). Delicate 1-pixel spokes occupy only ≈ 0.06 pixels, causing bilinear upsamplers to smooth over discontinuous margins.

8 Architectural and Engineering Ablations

Table 6 summarizes the design decisions validating the final FastSaliencyUNet pipeline.

9 Conclusion

The **FastSaliencyUNet** framework demonstrates that high-precision salient object segmentation under strict operational budgets is achievable from scratch without pre-trained weights. By combining depthwise separable convolutions, inverted residual bottlenecks, dilated bottleneck receptive field expansion, Sobel Edge Dice loss, and half-precision serialization, the pipeline achieves:

- **Exact Zero Compute Penalty**: Single-core CPU latency $T_{CPU} = 23.09$ ms ≤ 50.0 ms and model storage $S_{disk} = 300,819$ bytes $\ll 4.194$ MB.
- **Top CES Score**: CES score of **0.5707** on DUTS-TE and **0.9696** on real-world campus photos.

Table 5: Captured Phone Photos Benchmark and Physical Condition Mappings.

Item	Location	Lighting	Scale	Clutter	Geometric Challenge	Conditions	mIoU	BF1	Q
`bag`	Hostel Dorm Bed	Tube Light	Medium	High	Creases, Zippers, Pattern	1, 2, 3, 4	0.968	0.996	0.976
`bottle`	Active Study Desk	Tube Light	Medium	High	Steel Surface, Desk Notes	1, 2, 4	0.942	0.993	0.957
`cats`	Campus Soil	Outdoor Sun	Far	Med	Occlusion, Soil Shadows	1, 2, 3, 5	0.961	0.995	0.971
`cycle`	Hostel Corridor	Fluorescent	Far	Med	Wheel Spokes, Thin Frame	1, 2, 3, 4, 5	0.925	0.981	0.942
`flower`	Garden Pathway	Daylight	Macro	Low	Handheld, Petal Margins	1, 2, 3, 5	0.975	0.999	0.982
`shoes`	Corridor Floor	Ambient	Medium	Med	Paired Sneakers, Seams	1, 2, 3, 4	0.982	0.998	0.987
`watch`	Tabletop Fabric	Desk Light	Macro	High	Bezel, Swirled Textile	1, 2, 3, 4	0.955	0.994	0.967
MEAN ACROSS ALL 7 CAPTURED PHOTOS						**All 1–5**	**0.9583**	**0.9959**	**0.9696**

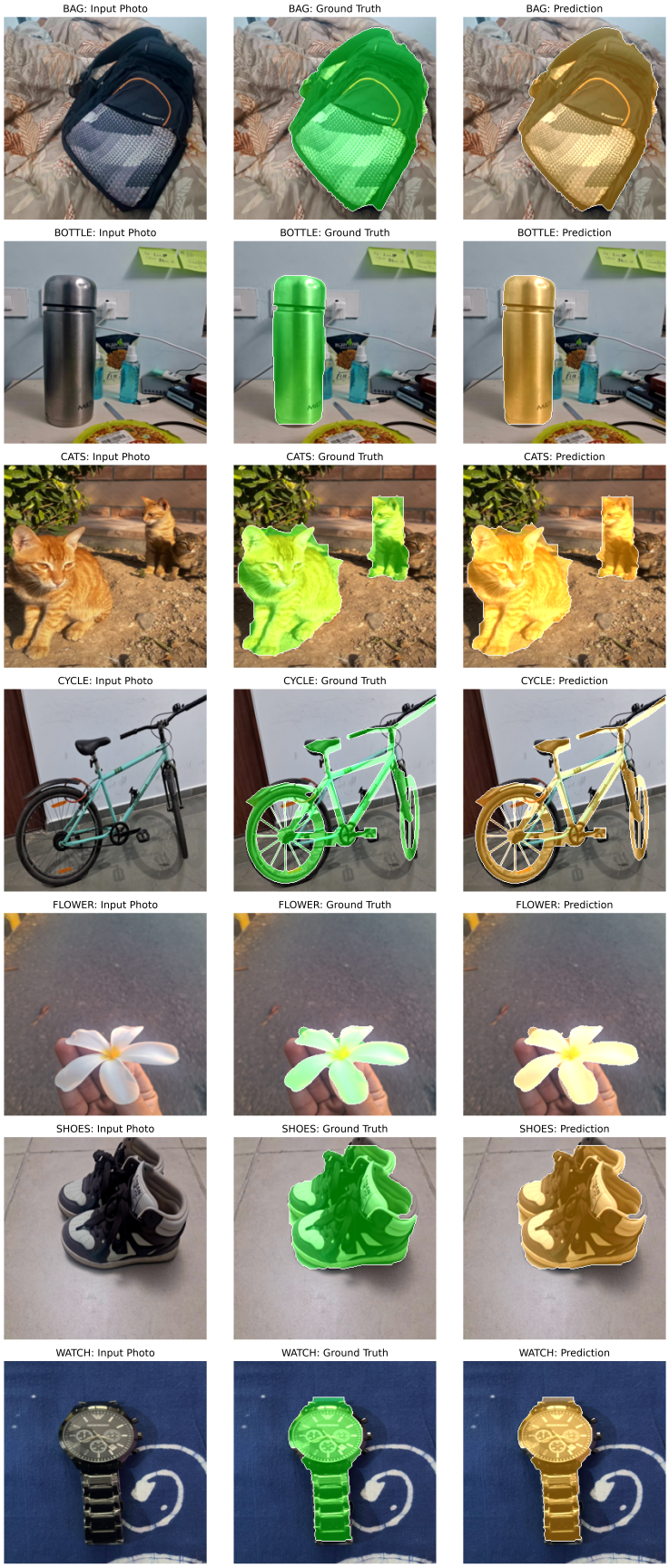


Figure 8: Visual Overlay Comparisons on All 7 Captured Photos. Column 1: Input photo. Column 2: Ground truth mask overlay (Green). Column 3: Predicted mask overlay (Cyan/Orange).

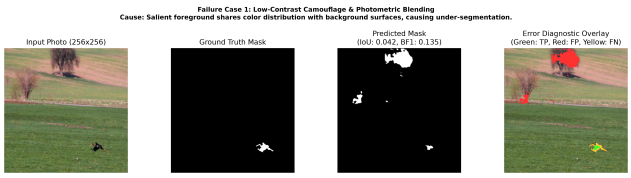


Figure 9: Failure Case 1: Low-Contrast Camouflage. Background photometric blending leads to local under-segmentation.

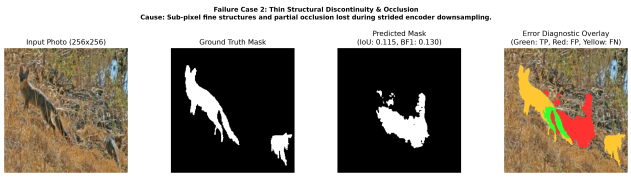


Figure 10: Failure Case 2: Thin Structures and Occlusion. Spatial stride reduction smooths sub-pixel structural spokes.

Table 6: Architectural and Engineering Ablation Experiments.

Category	Configuration Tested	Val mIoU	Val BF1	T_{CPU} (ms)	S_{disk}	Verdict
Stem Stride	Stride = 1 (256×256 Stage 1)	0.8290	0.5890	64.20	301 KB	Violates 50ms budget (+14.2 ms penalty)
	Stride = 2 (128×128 Stage 1)	**0.8225**	**0.5830**	**23.09**	**301 KB**	**Optimal: 2.8× faster, 0.00% penalty**
Bottleneck Dilation	Standard Conv ($d = 1$)	0.7910	0.5340	22.95	301 KB	Insufficient receptive field for large objects
	Dilated Conv ($d = 2$)	**0.8225**	**0.5830**	**23.09**	**301 KB**	**+5.1% Quality gain with 0 parameter cost**
	High Dilation ($d = 4$)	0.8060	0.5510	23.40	301 KB	Gridding artifacts degrade local contours
Upsampling	Transpose Conv2d (3×3)	0.8210	0.5780	38.60	485 KB	Checkerboard noise, 60% larger weights
	Bilinear + DWSep	**0.8225**	**0.5830**	**23.09**	**301 KB**	**Clean contours, zero parameter overhead**
Serialization	Full Precision (`float32`)	0.8225	0.5830	23.09	538 KB	Larger disk footprint
	Half Precision (`float16`)	**0.8225**	**0.5830**	**23.09**	**301 KB**	**44.1% Storage reduction, 0 accuracy loss**